R1CS 정의

개요

R1CS(Rank-1 Constraint System)는 산술 회로를 제약 조건의 집합으로 표현하는 방법입니다. zkSNARK 시스템에서 계산의 정확성을 검증하는 기본 구조를 제공합니다.

수학적 정의

기본 형태

R1CS 제약 조건은 다음과 같은 형태를 가집니다:

$$\langle A, X\rangle * \langle B, X\rangle = \langle C, X\rangle$$

여기서:

- **X**: 변수 벡터 $(x_1, x_2, \ldots, x_n)$
- **A, B, C**: 계수 벡터
- $\langle \bullet,\bullet \rangle$: 내적 연산

구체적 표현

$$(\Sigma_i\ a_i x_i) * (\Sigma_i\ b_i x_i) = \Sigma_i\ c_i x_i$$

선형 결합 (Linear Combination)

각 제약 조건의 좌변과 우변은 변수들의 선형 결합으로 표현됩니다. libiop에서는 이를 `linear_combination` 클래스로 구현합니다:

기본 구조

```
template<typename FieldT>
class linear_combination {
public:
std::vector<linear_term<FieldT>> terms; // 선형 항들의 벡터
// 생성자들
linear_combination() {};
linear_combination(const integer_coeff_t int_coeff);
linear_combination(const FieldT &field_coeff);
linear_combination(const variable<FieldT> &var);
linear_combination(const linear_term<FieldT> &lt);
// 항 추가 메서드들
void  add_term(const variable<FieldT> &var);
void  add_term(const variable<FieldT> &var, const integer_coeff_t int_coeff);
void  add_term(const variable<FieldT> &var, const FieldT &field_coeff);
void  add_term(const linear_term<FieldT> &lt);
// 변수 할당에 대한 평가
FieldT evaluate(const std::vector<FieldT> &assignment) const;
// 연산자 오버로딩
linear_combination<FieldT> operator+(const linear_combination<FieldT> &other) co
linear_combination<FieldT> operator-(const linear_combination<FieldT> &other) co
```

```
linear_combination<FieldT> operator*(const FieldT &field_coeff) const;

};
```

선형 항 (Linear Term)

각 선형 결합은 선형 항들의 집합으로 구성됩니다:

```
template<typename FieldT>

class linear_term {

public:

var_index_t index_; // 변수 인덱스

FieldT coeff_; // 계수

linear_term() {};

linear_term(const variable<FieldT> &var);

linear_term(const variable<FieldT> &var, const FieldT &field_coeff);

// 연산자 오버로딩

linear_term<FieldT> operator*(const FieldT &field_coeff) const;

linear_combination<FieldT> operator+(const linear_combination<FieldT> &other) co

};
```

변수 (Variable)

개별 변수는 인덱스로 표현됩니다:

```
template<typename FieldT>
```

```
class variable {

public:

var_index_t index_; // 변수 인덱스 (0은 상수 1 예약)

variable(const var_index_t index = 0) : index_(index) {};

// 계수와의 곱셈으로 선형 항 생성

linear_term<FieldT> operator*(const FieldT &field_coeff) const;

// 다른 선형 결합과의 덧셈/뺄셈

linear_combination<FieldT> operator+(const linear_combination<FieldT> &other) co

linear_combination<FieldT> operator-(const linear_combination<FieldT> &other) co

};
```

R1CS 제약 조건 클래스

기본 구조

libiop에서 R1CS 제약 조건은 `r1cs_constraint` 클래스로 구현됩니다:

```
template<typename FieldT>

class r1cs_constraint {

public:

linear_combination<FieldT> a_, b_, c_; // A, B, C 선형 결합

// 생성자들

r1cs_constraint() {};

r1cs_constraint(const linear_combination<FieldT> &a,
```

```
const linear_combination<FieldT> &b,

const linear_combination<FieldT> &c);

// 초기화 리스트를 사용한 생성자

r1cs_constraint(const std::initializer_list<linear_combination<FieldT>> &A,

const std::initializer_list<linear_combination<FieldT>> &B,

const std::initializer_list<linear_combination<FieldT>> &C);

// 비교 연산자

bool  operator==(const r1cs_constraint<FieldT> &other) const;

};
```

제약 조건 검증

제약 조건의 만족 여부는 다음과 같이 확인됩니다:

```
// r1cs_constraint_system 클래스 내부

template<typename FieldT>

bool r1cs_constraint_system<FieldT>::is_satisfied(

const r1cs_variable_assignment<FieldT> &full_variable_assignment

) const {

for (size_t c = 0; c < constraints_.size(); ++c) {

const FieldT ares = constraints_[c].a_.evaluate(full_variable_assignment);

const FieldT bres = constraints_[c].b_.evaluate(full_variable_assignment);

const FieldT cres = constraints_[c].c_.evaluate(full_variable_assignment);
```

```
if (!(ares * bres == cres)) {

return false; // 제약 조건 위반

}

}

return true;

}
```

디버깅 지원

디버그 모드에서는 제약 조건 위반 시 상세한 정보를 출력합니다:

```
#ifdef  DEBUG

auto it = constraint_annotations_.find(c);

printf("constraint %zu (%s) unsatisfied\n", c,

(it == constraint_annotations_.end() ? "no annotation" : it->second.c_str()));

printf("<a,(1,x)> = ");  ares.print();

printf("<b,(1,x)> = ");  bres.print();

printf("<c,(1,x)> = ");  cres.print();

dump_r1cs_constraint(constraints_[c], full_variable_assignment, variable_annotat

#endif
```

산술 회로에서 R1CS로의 변환

기본 게이트 변환

덧셈 게이트: z = x + y

```
template<typename FieldT>
R1CSConstraint<FieldT> create_addition_constraint(
size_t  x_index,
size_t  y_index,
size_t  z_index
) {
LinearCombination<FieldT> a, b, c;
// (x + y) * 1 = z
a.add_term(x_index, FieldT::one());
a.add_term(y_index, FieldT::one());
b.add_constant(FieldT::one()); // 상수 1
c.add_term(z_index, FieldT::one());
return  R1CSConstraint<FieldT>(a, b, c);
}
```

곱셈 게이트: z = x * y

```
template<typename FieldT>
R1CSConstraint<FieldT> create_multiplication_constraint(
size_t  x_index,
size_t  y_index,
size_t  z_index
```

```
) {
LinearCombination<FieldT> a, b, c;
// x * y = z
a.add_term(x_index, FieldT::one());
b.add_term(y_index, FieldT::one());
c.add_term(z_index, FieldT::one());
return  R1CSConstraint<FieldT>(a, b, c);
}
```

상수 곱셈: z = k * x

```
template<typename FieldT>
R1CSConstraint<FieldT> create_scalar_multiplication_constraint(
size_t  x_index,
size_t  z_index,
const FieldT&  scalar
) {
LinearCombination<FieldT> a, b, c;
// x * k = z
a.add_term(x_index, FieldT::one());
b.add_constant(scalar);
c.add_term(z_index, FieldT::one());
return  R1CSConstraint<FieldT>(a, b, c);
```

```
}
```

복합 연산 변환

제곱 연산: $z = x^2$

```
template<typename FieldT>
R1CSConstraint<FieldT> create_square_constraint(
size_t  x_index,
size_t  z_index
) {
LinearCombination<FieldT> a, b, c;
// x * x = z
a.add_term(x_index, FieldT::one());
b.add_term(x_index, FieldT::one());
c.add_term(z_index, FieldT::one());
return  R1CSConstraint<FieldT>(a, b, c);
}
```

역원 연산: $z = x^{-1}$ $(x \neq 0)$

```
template<typename FieldT>
R1CSConstraint<FieldT> create_inverse_constraint(
size_t  x_index,
size_t  z_index
```

```
) {
LinearCombination<FieldT> a, b, c;
// x * z = 1
a.add_term(x_index, FieldT::one());
b.add_term(z_index, FieldT::one());
c.add_constant(FieldT::one());
return  R1CSConstraint<FieldT>(a, b, c);
}
```

변환 예시

다항식 평가: $P(x) = x^3 + 2x^2 + 3x + 4$

단계별 분해

1. $t_1 = x * x$ (x^2)
2. $t_2 = t_1 * x$ (x^3)
3. $t_3 = 2 * t_1$ ($2x^2$)
4. $t_4 = 3 * x$ ($3x$)
5. $t_5 = t_2 + t_3$ ($x^3 + 2x^2$)
6. $t_6 = t_5 + t_4$ ($x^3 + 2x^2 + 3x$)
7. $y = t_6 + 4$ ($x^3 + 2x^2 + 3x + 4$)

R1CS 제약 조건들

```
template<typename FieldT>
vector<R1CSConstraint<FieldT>> create_polynomial_constraints(
size_t  x_index,
size_t  y_index,
size_t  t1_index, // x²
size_t  t2_index, // x³
size_t  t3_index, // 2x²
size_t  t4_index, // 3x
size_t  t5_index, // x³ + 2x²
size_t  t6_index  // x³ + 2x² + 3x
) {
vector<R1CSConstraint<FieldT>> constraints;
// t₁  = x * x
constraints.push_back(create_multiplication_constraint(x_index, x_index, t1_inde
// t₂  = t₁  * x
constraints.push_back(create_multiplication_constraint(t1_index, x_index, t2_ind
// t₃  = 2 * t₁
constraints.push_back(create_scalar_multiplication_constraint(
t1_index, t3_index, FieldT(2)));
// t₄  = 3 * x
constraints.push_back(create_scalar_multiplication_constraint(
x_index, t4_index, FieldT(3)));
```

```
// t5 = t2  + t3

constraints.push_back(create_addition_constraint(t2_index, t3_index, t5_index));

// t6 = t5 + t4

constraints.push_back(create_addition_constraint(t5_index, t4_index, t6_index));

// y = t6 + 4

LinearCombination<FieldT> a, b, c;

a.add_term(t6_index, FieldT::one());

a.add_constant(FieldT(4));

b.add_constant(FieldT::one());

c.add_term(y_index, FieldT::one());

constraints.push_back(R1CSConstraint<FieldT>(a, b, c));

return constraints;

}
```

최적화 기법

제약 조건 최소화

```
template<typename FieldT>

class ConstraintOptimizer {

public:

// 중복 제약 조건 제거

vector<R1CSConstraint<FieldT>> remove_duplicates(
```

```
const vector<R1CSConstraint<FieldT>>&  constraints

) const {

set<string> seen_constraints;

vector<R1CSConstraint<FieldT>> unique_constraints;

for (const  auto& constraint : constraints) {

string constraint_str = constraint.to_canonical_string();

if (seen_constraints.find(constraint_str) == seen_constraints.end()) {

seen_constraints.insert(constraint_str);

unique_constraints.push_back(constraint);

}

}

return unique_constraints;

}

// 선형 결합 최적화

LinearCombination<FieldT> optimize_linear_combination(

const LinearCombination<FieldT>&  lc

) const {

LinearCombination<FieldT> optimized;

// 계수가 0인 항 제거는 이미 add_term에서 처리됨

// 추가적인 최적화 로직 구현 가능

return lc;

}
```

```
};
```

희소 행렬 표현

```
template<typename FieldT>
class SparseR1CSMatrix {
private:
vector<vector<pair<size_t, FieldT>>> rows; // 각 행의 (인덱스, 값) 쌍들
size_t num_variables;
public:
SparseR1CSMatrix(size_t  num_vars) : num_variables(num_vars) {}
// 제약 조건 추가
void  add_constraint(const R1CSConstraint<FieldT>&  constraint) {
auto a_sparse = constraint.get_a().to_sparse();
auto b_sparse = constraint.get_b().to_sparse();
auto c_sparse = constraint.get_c().to_sparse();
rows.push_back(a_sparse.first, a_sparse.second);
rows.push_back(b_sparse.first, b_sparse.second);
rows.push_back(c_sparse.first, c_sparse.second);
}
// 메모리 효율적인 행렬-벡터 곱셈
vector<FieldT> multiply(const vector<FieldT>&  vec) const {
vector<FieldT> result(rows.size(), FieldT::zero());
```

```
#pragma omp parallel for
for (size_t i = 0; i < rows.size(); i++) {
for (const auto& term : rows[i]) {
size_t col = term.first;
FieldT coeff = term.second;
if (col < vec.size()) {
result[i] += coeff * vec[col];
}
}
}
return result;
}
};
```

libiop 구현 세부사항

타입 정의

libiop에서는 다음과 같은 타입 별칭을 사용합니다:

```
typedef size_t var_index_t; // 변수 인덱스 타입
typedef long integer_coeff_t; // 정수 계수 타입

template<typename FieldT>
```

```
using r1cs_primary_input = std::vector<FieldT>; // 공개 입력


template<typename FieldT>

using r1cs_auxiliary_input = std::vector<FieldT>; // 보조 입력


template<typename FieldT>

using r1cs_variable_assignment = std::vector<FieldT>; // 전체 변수 할당
```

변수 인덱싱 규칙

- **인덱스 0**: 항상 상수 1로 예약됨
- **인덱스 1 ~ primary_input_size**: 공개 입력 변수들
- **인덱스 (primary_input_size + 1) ~ num_variables**: 보조 변수들

선형 결합 평가 과정

```
template<typename FieldT>

FieldT linear_combination<FieldT>::evaluate(const std::vector<FieldT> &assignmen

FieldT result = FieldT::zero();

for (const linear_term<FieldT> &term : terms) {

if (term.index_ < assignment.size()) {

result += term.coeff_ * assignment[term.index_];

}

}

return result;
```

```
}
```

초기화 리스트 생성자

복잡한 제약 조건을 쉽게 생성할 수 있도록 초기화 리스트를 지원합니다:

```
// 예시: (x + y) * (z + w) = (u + v)
r1cs_constraint<FieldT> constraint(
{x + y}, // A = x + y
{z + w}, // B = z + w
{u + v} // C = u + v
);
```

이 생성자는 여러 선형 결합을 하나로 합치는 기능을 제공합니다:

```
template<typename FieldT>
r1cs_constraint<FieldT>::r1cs_constraint(
const std::initializer_list<linear_combination<FieldT>> &A,
const std::initializer_list<linear_combination<FieldT>> &B,
const std::initializer_list<linear_combination<FieldT>> &C
) {
// 각 리스트의 모든 항들을 해당 선형 결합에 추가
for (auto lc_A : A) {
a_.terms.insert(a_.terms.end(), lc_A.terms.begin(), lc_A.terms.end());
}
```

```
for (auto lc_B : B) {
b_.terms.insert(b_.terms.end(), lc_B.terms.begin(), lc_B.terms.end());
}
for (auto lc_C : C) {
c_.terms.insert(c_.terms.end(), lc_C.terms.begin(), lc_C.terms.end());
}
}
```

다음 단계

다음 문서에서는 제약 시스템을 통해 R1CS 제약 조건들의 생성과 검증 과정을 자세히 살펴보겠습니다.

← R1CS 관계 목차 | 다음: 제약 시스템 →